

Análisis y Diseño con Patrones

PEC 1: Principios de Diseño y Patrones de Análisis

Pregunta 1 (20%)

Supongamos que estamos desarrollando un software que implementa un sistema de red social para competir con las grandes corporaciones actuales. La red social se vertebrará alrededor de comunidades, a las que los usuarios pueden unirse en una fecha concreta como miembros. Un usuario es miembro de la comunidad durante el periodo que desee, pudiendo abandonarla a su voluntad. En caso de que un usuario haya abandonado una comunidad, puede volver a unirse de nuevo posteriormente si así lo desea.

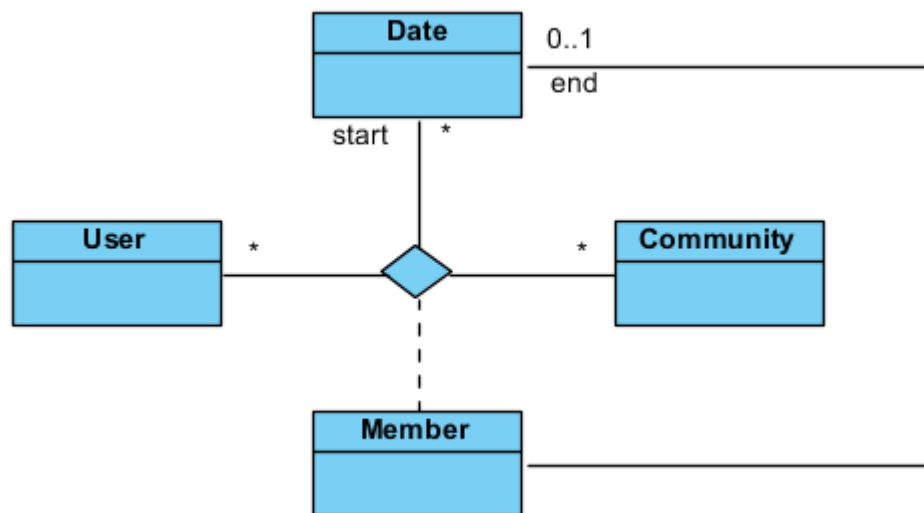
Nos proponemos desarrollar el módulo que implementa esta red social. Antes de hacer el diagrama estático de análisis, sin embargo, revisemos los patrones de análisis buscando uno que pueda ayudar a resolver el problema comentado.

Indica qué patrón de análisis habría que tener especialmente en cuenta y justifica brevemente por qué. Propón un diagrama estático de análisis donde se aplique el patrón de análisis identificado para los requisitos descritos.

Nota: Si es necesario, puedes hacer alguna variación sobre el patrón tal y como está explicado en los materiales para poder satisfacer todos los requisitos enumerados.

Solución

En este caso podemos aplicar el patrón “Asociación histórica” ya que nos permite mantener un histórico de membresía de los usuarios en las comunidades, en distintos rangos de fechas.



Hemos asignado una fecha de fin en la clase asociativa para poder contrastar con las fechas de inicio, de modo que podamos controlar y evitar solapamientos entre periodos de membresía de un usuario en una misma comunidad.

Pregunta 2 (20%)

Imaginemos que estamos desarrollando un software de *fintech* que permita a los usuarios mantener cuentas bancarias en distintas divisas. Cada cuenta bancaria dispone de un saldo especificado en una divisa en concreto de las disponibles en el sistema.

Específicamente, el subconjunto de divisas que queremos contemplar es el siguiente:

- Dólar (USD)
- Euro (EUR)
- Libra esterlina (GBP)

Nos gustaría que, en un futuro, se pudiera añadir otras divisas sin tener que hacer grandes cambios en el sistema. También nos interesa poder ofrecer a los usuarios la posibilidad de cambiar de divisa

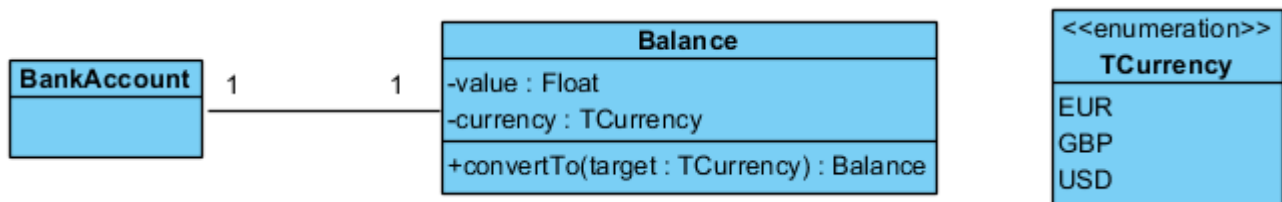
su cuenta bancaria, de modo que necesitamos un método para convertir el saldo desde una divisa a otra.

Indica qué patrón de análisis habría que tener especialmente en cuenta y justifica brevemente por qué. Propón un diagrama estático de análisis donde se aplique el patrón de análisis identificado en los requisitos descritos.

Nota: Si es necesario, puedes hacer alguna variación sobre el patrón tal y como está explicado en los materiales para poder satisfacer todos los requisitos enumerados.

Solución

En este caso podemos aplicar el patrón "Cantidad" ya que queremos que la unidad de medida sea explícita y poder utilizar diferentes unidades de medida, así como poder hacer conversiones fácilmente (es decir, nos encontramos ante el problema que resuelve este patrón).



Hemos aplicado una variación respecto a los ejemplos de los materiales donde se tiene en cuenta el hecho de que queremos poder convertir entre divisas. Para ello hemos incluido un método en la clase *Balance* para, dada una divisa destino, obtener el nuevo saldo. Además, hemos creado una enumeración *TCurrency* para concretar los valores posibles de las divisas aceptadas. Una solución con el atributo *currency* con tipo de datos *String* también sería válida.

Otra solución posible sería especificar *Currency* como clase aparte, incluyendo el método de conversión de divisas, de la siguiente forma:



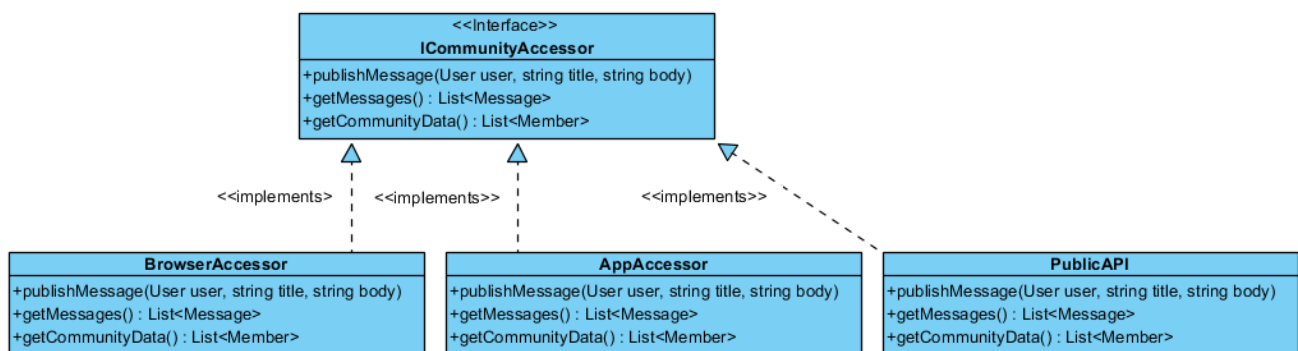
De esta forma, concretamos más la responsabilidad del tipo de cambio a la divisa, en lugar de tenerla agregada en el saldo.

Pregunta 3 (30%)

Uno de los principios de diseño que debe cumplir un sistema para obtener un nivel alto de calidad es el principio de "Segregación de Interfaces" que dice que "Los clientes no tendrían que depender de operaciones que no utilizan".

Supongamos ahora que estamos diseñando el sistema de red social de la Pregunta 1. En concreto, hemos de contemplar las típicas operaciones de publicación de mensaje en una comunidad y la consulta de mensajes publicados en las mismas. En el método de publicación de mensaje necesitamos indicar el usuario que realiza la publicación, el título de la misma y el cuerpo del mensaje. Tenemos un método de consulta de mensajes publicados en una comunidad que retorna el listado de mensajes (incluyendo fecha de publicación, nombre y avatar del autor, título y cuerpo). Adicionalmente, se puede solicitar el listado completo de miembros de la comunidad (incluyendo nombre, avatar y última fecha de alta de membresía).

En un diagrama de clases, podríamos representarlo como:



La red social está disponible para su acceso tanto desde navegadores de dispositivos desktop (Windows y Mac) como desde sendas apps para dispositivos móviles Android y iOS. En los navegadores para Windows y Mac se mostrará la información completa devuelta por el método de consulta de mensajes y por el método de consulta de miembros de la comunidad. En cambio, en las apps tendremos una visión más reducida para no malgastar datos y por ello el listado de mensajes de una comunidad sólo muestra los datos de dichos mensajes, obviando el método que proporciona el listado adicional con todos los miembros de la comunidad.

Nuestra red social expone públicamente una API mediante la cual cualquier sistema externo puede obtener los mensajes publicados en una comunidad, sin la información adicional de sus miembros, como hacen nuestras apps. Pero en esta API pública no queremos exponer el método de publicación de mensaje.

- a) Este ejemplo constituye o puede constituir una violación del principio de segregación de interfaces. ¿Por qué?
- b) ¿Qué consecuencias tendría esta violación?

Solución

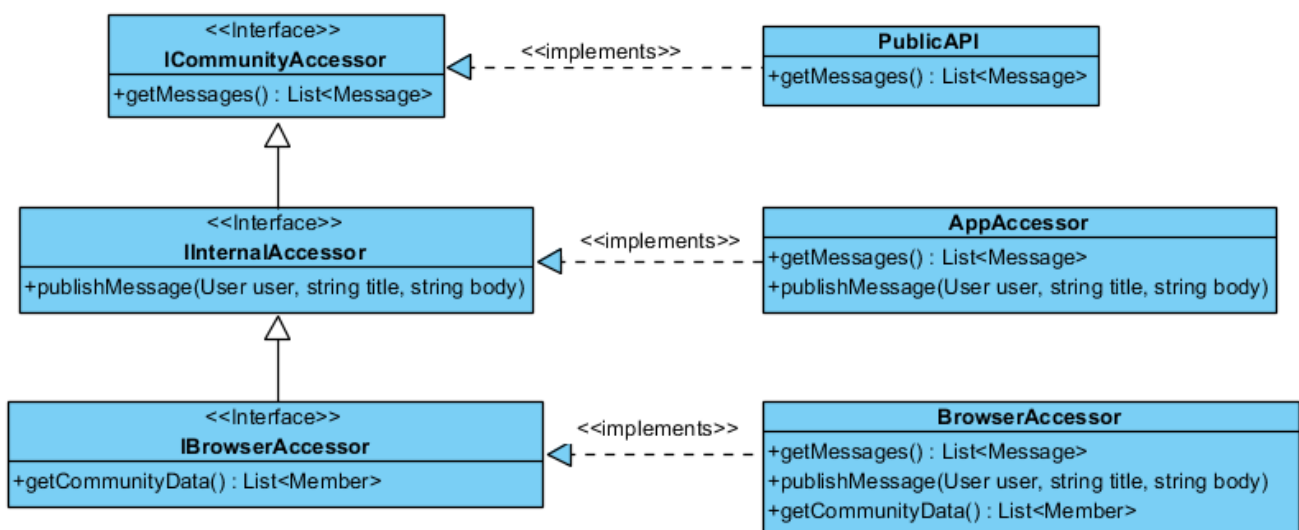
- a) Se trata de una violación del principio de segregación de interfaces ya que para mostrar el listado de mensajes desde cualquiera de las apps estamos dependiendo de información que no utilizamos, como es el listado completo de miembros de la comunidad consultada. Este método, por tanto, no tiene por qué implementarse para las apps. Por otra parte, la API necesita implementar una excepción para el método de publicación de mensaje, cuando en realidad no permitimos a sistemas externos realizar esta acción.
- b) La principal consecuencia es que será difícil hacer evolucionar el sistema ya que cada vez que queramos añadir datos a devolver a los navegadores de Windows y Mac podremos afectar a la implementación de las apps (para que su UI ignore estos nuevos datos). Además, cualquier método que queramos añadir a nuestro sistema tendrá su correspondiente implementación en la API, de modo que (1) cualquier cambio en nuestra funcionalidad afectará a la integración de sistemas externos; y (2) potencialmente estaremos exponiendo métodos que no queremos que se conozcan públicamente, aun cuando la implementación se limite al lanzamiento de una excepción.

Pregunta 4 (30%)

Propón cómo se podría diseñar el sistema anterior para que se satisfaga el principio de segregación de interfaces.

Usa diagramas estáticos de análisis, diagramas de secuencia y / o pseudocódigo de operaciones según creas más relevante para explicar la solución propuesta. Razona, también, cómo la solución supera las consecuencias negativas del apartado *b* de la pregunta anterior.

Solución



Como podemos ver en esta solución, y gracias a la segregación de interfaces, cada una de las clases que proporcionan acceso a los datos de una comunidad implementa única y exclusivamente los métodos apropiados, son independientes, de forma que las desacoplamos. Como resultado, además, ninguna de las clases ha de implementar métodos vacíos.